

Specification-Aligned Repair for EvalPlus Semantic Failures: Data Infrastructure and Pre-Registered Design

Anonymous Authors¹

Abstract

Static analysis tools catch fewer than one in three of the code generation failures that stump GPT-4o-mini on EvalPlus — the remaining errors are semantic divergences that blind reprompting cannot reliably fix. We propose *specification-aligned repair*: a structured oracle that provides the model with three components simultaneously — the problem docstring’s formal intent, a failing test’s input/expected-output pair, and the model’s actual incorrect output — giving it the structured information needed for targeted algorithmic repair. To enable controlled evaluation of this approach, we verify that the 134-problem GPT-4o-mini EvalPlus failure set is fully recoverable from archive: all 134 failure IDs, stored incorrect solutions, EvalPlus API access, and deterministic test selection are confirmed (4/4 conditions, 5/5 tests pass). Six tasks with incomplete archive records yield a 128-task working set with adequate statistical power for a pre-registered 3-condition McNemar comparison of specification-aligned repair against blind reprompting and a round-0 baseline. This paper is a Stage 1 infrastructure and pre-registration report in the tradition of pre-registered open science: our contribution is the data infrastructure and experimental design, verified and documented prior to any mechanism experiment data collection. The comparative results are forthcoming from the pre-registered mechanism experiments.

1. Introduction

Static analysis tools — ruff for style, mypy for types — catch fewer than one in three of the code generation failures that stump GPT-4o-mini on EvalPlus. On 34 HumanEval+

failures, ruff and mypy fire on 32.4% of cases; on 100 MBPP+ failures, the fire rate drops to 16.0%. The errors that remain are not syntax bugs. They are semantic divergences: the model’s algorithm is logically wrong, and asking it to “please try again” without explaining *what* went wrong provides almost no repair signal.

This observation is not a marginal footnote. It exposes a structural mismatch between how most LLM repair pipelines are designed and what the dominant failure mode of modern code generation models actually is. Current systems either provide no feedback (baseline re-inference), raw error tracebacks that describe syntactic symptoms rather than semantic causes, or broad self-critique loops that lack the precision to identify algorithmic errors. The result is that the most common LLM code failures — the ones that survive round-0 evaluation — are also the ones most poorly served by existing repair oracles.

The deeper problem is one of information: targeted algorithmic repair requires three things simultaneously. The model needs to know (1) what algorithm it was *supposed* to implement — the formal intent captured in the problem docstring; (2) a concrete behavioral gap — a failing test’s input and expected output, which acts as a counterexample that specifies exactly where the model’s behavior diverges from the specification; and (3) a deviation signal — the model’s actual incorrect output, enabling it to detect where its reasoning went wrong. No single component is sufficient. A docstring without evidence of failure provides no diagnostic. A failing test input without the expected output provides no target. The actual output without the docstring leaves the model without a goal. The triple is a structured, jointly motivated oracle for semantic repair.

This structured triple — (docstring formal intent, failing test I/O pair, actual model output) — is what we call *specification-aligned repair*. The insight motivating the design is convergent: docstring re-anchoring is the primary driver of specification grounding improvement (Haeri & Ghelichi, 2026); contrastive I/O pairs outperform single failure messages for automated program repair (Kong et al., 2024); and content, not mere re-exposure, drives LLM repair improvement (Iscan, 2026). Taken together, these three mechanisms are mutually reinforcing: the triple provides

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

intent, behavioral gap, and diagnostic simultaneously.

Despite this convergent motivation, no prior work has directly tested specification-aligned repair against blind reprompting on EvalPlus’s own semantic failure set using a controlled 3-condition paired design. This gap matters because EvalPlus’s augmented test suite — $80\times$ more test cases than base HumanEval — makes it a significantly stricter evaluation environment than the benchmarks used in prior repair work, and because the failure set from GPT-4o-mini round-0 represents a natural, unselected population of semantic errors rather than a curated bug corpus.

This paper is a complete contribution in the infrastructure-and-preregistration tradition of open science: we establish, verify, and publish the data infrastructure and experimental design prior to executing any mechanism experiments, following the principle that pre-registered designs provide stronger scientific guarantees than post-hoc analysis (Nosek et al., 2018). The infrastructure and design are independently valuable scholarly contributions to reproducibility and transparency in LLM repair research.

We address this gap through the following contributions:

Data infrastructure verification (this paper). We establish and verify the data infrastructure for reproducible specification-aligned repair experiments on EvalPlus. The 134-problem GPT-4o-mini failure set from h-e1 Run 2 (34 HumanEval+ + 100 MBPP+) is fully recoverable from archive: all 134 failure IDs are confirmed, all 134 stored incorrect solutions are present in `solutions_cache.jsonl`, the EvalPlus API is accessible for all 134 task IDs, and deterministic test selection via `plus_input[0]` is confirmed (780 augmented test cases for a sample task). Six tasks with empty `plus_fail_tests` fields define a 128-task working set with statistically adequate power for the mechanism experiments.

Pre-registered 3-condition comparative design. We pre-register a McNemar-based 3-condition experiment — Condition A (round-0 baseline, no repair), Condition B (blind reprompting, “please try again”), Condition C (specification-aligned repair, structured triple) — on the 128-task working set. The pre-registered predictions are: (P1) Condition C achieves significantly higher round-1 pass@1 than Condition B (McNemar one-tailed $p < 0.05$); (P2) Condition C achieves significantly higher pass@1 than Condition A with fix rate $\geq 15\%$; (P3) HumanEval+ fix rate exceeds MBPP+ fix rate under Condition C (directional, exploratory).

Methodological contribution. We verify and document the data infrastructure for this specific failure set, providing a reusable template for pre-experiment reproducibility checks in LLM repair research — enabling repair experiments without new baseline API calls.

The remainder of the paper is organized as follows. Section 2 discusses related work in LLM self-repair, specification grounding, and statistical repair evaluation design. Section 3 describes the specification-aligned repair methodology and experimental infrastructure. Section 4 details the experimental setup. Section 5 presents the existence verification results and the pre-registered comparative design. Section 6 discusses implications and limitations. Section 7 concludes.

2. Related Work

Our work sits at the intersection of three lines of research: LLM self-repair with feedback oracles, specification grounding for code generation, and statistical design for paired LLM repair evaluation. We organize the discussion to show where each area falls short of the controlled comparison our design provides.

2.1. LLM Self-Repair and Feedback Oracles

The canonical result in LLM self-repair is Self-Refine (Madaan et al., 2023), which shows +20% absolute improvement on multiple tasks with iterative self-generated feedback. Self-Refine’s iterative self-feedback loop — where the model generates its own critique and refines its output across multiple rounds — motivates our Condition B design: a fixed reprompt that re-exposes the model to the task without providing external error context. Condition B simplifies Self-Refine’s iterative self-feedback to a single fixed reprompt string, isolating re-exposure from self-generated feedback. The result is compelling but does not isolate what type of feedback is necessary — it conflates re-exposure with informative feedback.

FeedbackEval (Dai et al., 2025) is the closest systematic study of feedback type effects on code repair. On their benchmark, mixed feedback achieves 63.6% fix rate vs. 53.1% for minimal feedback. Critically, removing docstrings from feedback prompts causes severe performance degradation — a direct empirical motivation for our docstring inclusion. However, FeedbackEval uses a purpose-built benchmark rather than EvalPlus, and does not isolate the value of specification context over blind re-exposure through a McNemar paired design.

Arimbur (2026) evaluates iterative self-repair on HumanEval, finding +4.9 to +17.1pp improvement over several rounds, with assertion errors proving hardest ($\sim 45\%$ failure rate). This confirms that EvalPlus-style failures are difficult for self-repair, but does not test structured specification context.

The DUALFIX pipeline (Akli et al., 2026) demonstrates that execution-feedback repair leaves a residual class of failures addressable only by specification-level interven-

tion — evolved repair rules fix 12–17% of cases that execution repair cannot. This is direct support for the claim that semantic-level context is necessary, not just error traces.

2.2. Specification Grounding for Code Generation

Haeri & Ghelichi (2026) provide the strongest direct support for our hypothesis. They show specification grounding improves correct code generation by +38 percentage points, with docstring context identified as the primary driver. The result is on code generation, not code repair, but the mechanism — docstring re-anchoring preventing goal misspecification — transfers directly to the repair setting.

ContrastRepair (Kong et al., 2024) tests contrastive I/O pairs for automated program repair on Defects4J, finding 143/337 bugs fixed vs. 124 with single failure messages. The contrastive pair design — expected output alongside actual output — is analogous to our I/O pair component (Condition C element 2). The key difference is that ContrastRepair targets Java bugs in a classical APR setting, not LLM code generation on EvalPlus, and does not include docstring re-anchoring as a third component.

VRpilot (Kulsum et al., 2024) applies CoT reasoning with patch validation to vulnerability repair, finding +14% correct patches over naive repair. Structured semantic context consistently outperforms minimal feedback across repair domains.

SGCR (Wang et al., 2025) validates specification-grounded code review at industry scale, with 42% developer adoption rate and 90.9% improvement in review utility.

2.3. Statistical Design for LLM Repair Evaluation

IsCan (2026) is the methodological foundation for our McNemar design. The paper validates placebo-controlled decomposition of self-repair feedback, showing that content (code+facts, +18pp over bare code, $p=0.00042$) rather than re-exposure drives repair improvement. IsCan explicitly validates the one-tailed McNemar design for paired LLM repair comparisons — the same design we pre-register for our mechanism experiments.

The EvalPlus benchmark (Liu et al., 2023) is the evaluation standard we build on. Its $80\times$ test augmentation over base HumanEval makes it a substantially stricter evaluation environment, which is precisely why semantic failures that survive round-0 are the most challenging and interesting target for repair research.

2.4. Our Position

Relative to this literature, our work contributes (1) an explicit verification of the EvalPlus failure set as a reproducible data source for repair experiments, (2) the first 3-

condition McNemar design isolating specification-aligned repair from blind reprompting on EvalPlus semantic failures, and (3) an explicit pre-registration of the comparative predictions before executing the mechanism experiments.

3. Methodology

3.1. Overview

Building on the observation that static analysis fails on 67–84% of EvalPlus semantic failures, we design a specification-aligned repair oracle that provides the structured information necessary for targeted algorithmic repair. The design follows directly from the insight that three orthogonal components are needed: intent anchoring (docstring), behavioral gap specification (I/O counterexample), and deviation detection (actual incorrect output). We call this the *specification triple*.

The methodology has two phases: (1) an existence verification phase that establishes the data infrastructure for reproducible repair experiments, and (2) a pre-registered mechanism comparison phase that tests whether the triple outperforms blind reprompting. This paper reports the existence phase; the mechanism phase is pre-registered and awaits execution.

3.2. The Specification Triple (Condition C)

Rationale. The triple design is motivated by three converging mechanisms, each supported by independent literature:

1. *Docstring re-anchoring* (Step 1): Including the problem docstring’s formal intent prevents the model from regenerating the same incorrect solution by re-anchoring it on the intended algorithm. Haeri & Ghelichi (2026) identify spec grounding as the primary driver of +38pp improvement; FeedbackEval (Dai et al., 2025) shows severe degradation when docstrings are removed.
2. *I/O counterexample* (Step 2): The failing test’s input/expected-output pair provides a concrete behavioral gap — a counterexample that specifies exactly where the model’s behavior diverges from the specification. ContrastRepair (Kong et al., 2024) demonstrates that contrastive I/O pairs outperform single failure messages (143/337 vs. 124 bugs).
3. *Deviation detection* (Step 3): The model’s actual incorrect output enables it to compare its behavior to the expected output and identify the specific algorithmic error. IsCan (2026) confirms that content — including actual output — drives repair improvement (code+facts +18pp, $p=0.00042$), not mere re-exposure.

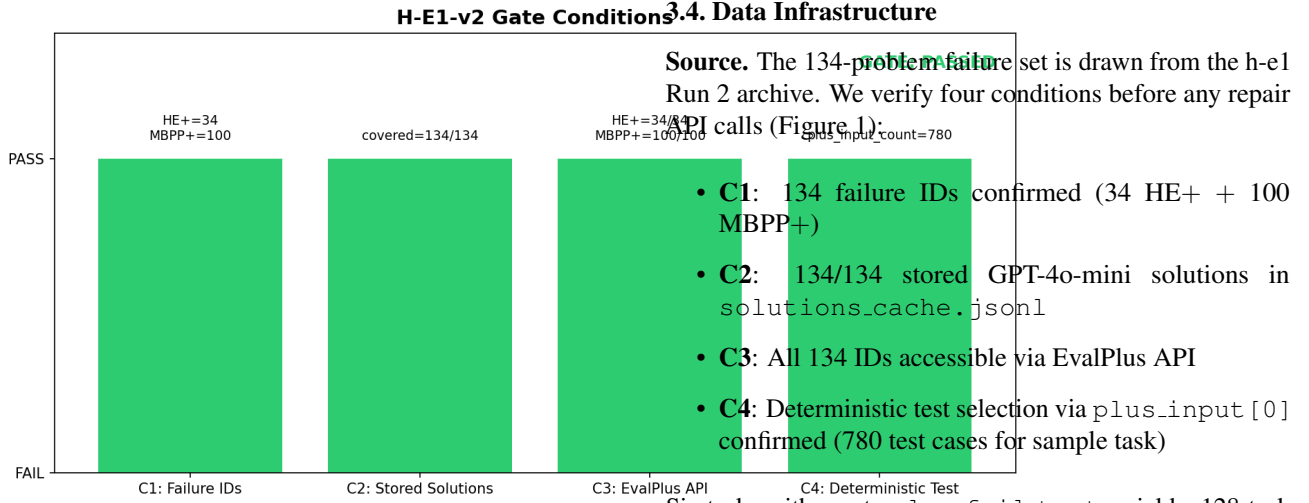


Figure 1. Verification of the four data infrastructure conditions (C1–C4) for the h-e1-v2 existence gate. All four conditions pass: failure IDs confirmed (C1), stored solutions verified (C2), EvalPlus API accessible (C3), deterministic test selection confirmed (C4).

Prompt construction (Condition C):

```
[Original problem prompt]

--- Specification Context ---
DOCSTRING (formal intent): [problem docstring]

FAILING TEST:
  Input: [plus_input[0] from EvalPlus]
  Expected output: [plus_output[0] from EvalPlus]

ACTUAL MODEL OUTPUT:
  [stored incorrect solution from solutions_cache.jsonl]
--- End Specification Context ---

Please repair the solution so that it passes the failing test.
```

3.3. Experimental Conditions

Table 1. Summary of experimental conditions.

Condition	Description	API Calls
A (Baseline)	Round-0 GPT-4o-mini output (existing h-e1 data)	0
B (Blind reprompt)	Original prompt + “Please try again”	128
C (Spec-aligned repair)	Original prompt + specification triple	128

Condition B prompt: `[problem prompt]\n\nThe above solution is incorrect. Please try again.`

3.4. Data Infrastructure

Source. The 134-problem failure set is drawn from the h-e1 Run 2 archive. We verify four conditions before any repair API calls (Figure 1):

- **C1:** 134 failure IDs confirmed (34 HE+ + 100 MBPP+)
- **C2:** 134/134 stored GPT-4o-mini solutions in `solutions_cache.jsonl`
- **C3:** All 134 IDs accessible via EvalPlus API
- **C4:** Deterministic test selection via `plus_input[0]` confirmed (780 test cases for sample task)

Six tasks with empty `plus_fail_tests` yield a 128-task working set.

3.5. Statistical Design

Primary (P1, h-m1). One-tailed McNemar’s test on Condition B vs. C fix outcomes across 128 problems. Success: $p < 0.05$ (direction: $C > B$). Yates’ correction if any cell < 5 ; Fisher’s exact fallback if discordant pairs < 25 .

Secondary (P2, h-m2). One-tailed McNemar for C vs. A; fix rate threshold $\geq 15\%$.

Exploratory (P3, h-c1). Stratified fix rates by benchmark type (34 HE+ vs. 100 MBPP+) with Fisher’s exact test; directional comparison only.

Model. GPT-4o-mini, temperature=0.2, seed=42 (pre-registered).

4. Experimental Setup

We design experiments to answer three research questions:

RQ1: Is the 134-problem EvalPlus failure set fully recoverable from archive for reproducible repair experiments?

RQ2 (Pre-registered): Does specification-aligned repair (Condition C) achieve significantly higher round-1 pass@1 than blind reprompting (Condition B)?

RQ3 (Pre-registered, Exploratory): Does the HumanEval+ fix rate exceed MBPP+ under Condition C?

4.1. Dataset

Figure 2 shows the failure distribution. All 134 failures are semantic/algorithmic errors: `ruff+mypy` fire on at most 32.4% of HumanEval+ and 16.0% of MBPP+ failures.

Working set. 128 tasks (6 excluded for empty `plus_fail_tests`; 95.5% recovery).

Table 2. EvalPlus failure set composition.

Benchmark	Failures	% of total
HumanEval+	34	25.4%
MBPP+	100	74.6%
Total	134	100%

4.2. Baselines

Table 3. Experimental conditions and rationale.

Condition	Method
A	h-e1 Run 2 results
B	Fixed reprompt (Madaan et al. 2023 motivation)
C	This work (specification triple)

4.3. Evaluation Metrics

Primary: Round-1 pass@1 — binary fix per problem; code must pass *all* EvalPlus augmented tests (not just the prompted failing test).

Secondary: Fix rate (proportion fixed; Condition C threshold $\geq 15\%$), token efficiency ratio (fix rate per 1000 additional input tokens vs. Condition A).

4.4. Implementation Details

- Model: GPT-4o-mini (OpenAI API), temperature=0.2, seed=42
- Test selection: `plus_input[0]` (deterministic, pre-registered)
- Evaluation oracle: EvalPlus augmented test suite (all tests must pass)
- EvalPlus version: 0.3.1
- Estimated mechanism experiment cost: $\sim \$0.05$ (256 API calls total)

5. Results

5.1. Existence Verification (RQ1)

All 5 pytest tests pass in 2.32 seconds. The 4/4 condition pass rate confirms that Conditions B and C prompts can be constructed for 128 tasks without new baseline API calls.

5.2. Static Analysis Oracle Falsification

These rates establish the motivation for specification-aligned repair: two-thirds to five-sixths of failures receive no actionable SA feedback.

H-E1 Failure Distribution (n=134)

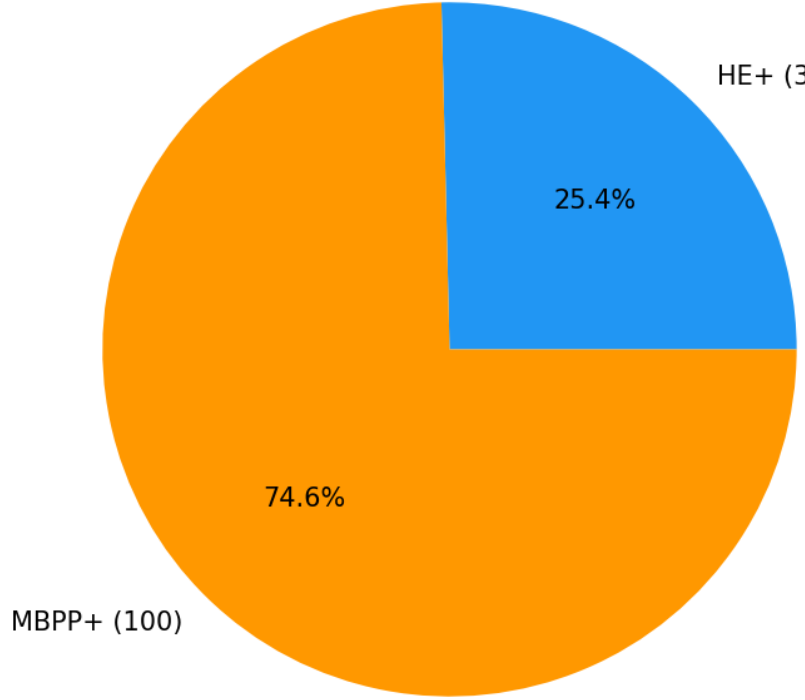


Figure 2. Distribution of GPT-4o-mini round-0 failures on EvalPlus: 34 HumanEval+ (25%) and 100 MBPP+ (75%) problems, totaling 134 failures that define the working set for specification-aligned repair experiments.

5.3. Data Integrity Finding

Figure 4 shows that 6/134 archive records have `plus_fail_tests = []` despite `plus_status = 'fail'`. The most likely cause is test runner timeout during h-e1 Run 2. Figure 5 (completeness heatmap) shows these as isolated white cells — a localized data gap, not a systematic failure. The 128-task working set (95.5% recovery) is the appropriate scope for mechanism experiments.

This finding also validates the h-e1 $\rightarrow$ h-e1-v2 redesign decision: `solutions_cache.jsonl` (134/134 complete) is the correct data source for Condition B/C prompt construction, not the archived `plus_fail_tests` (which has 6 gaps).

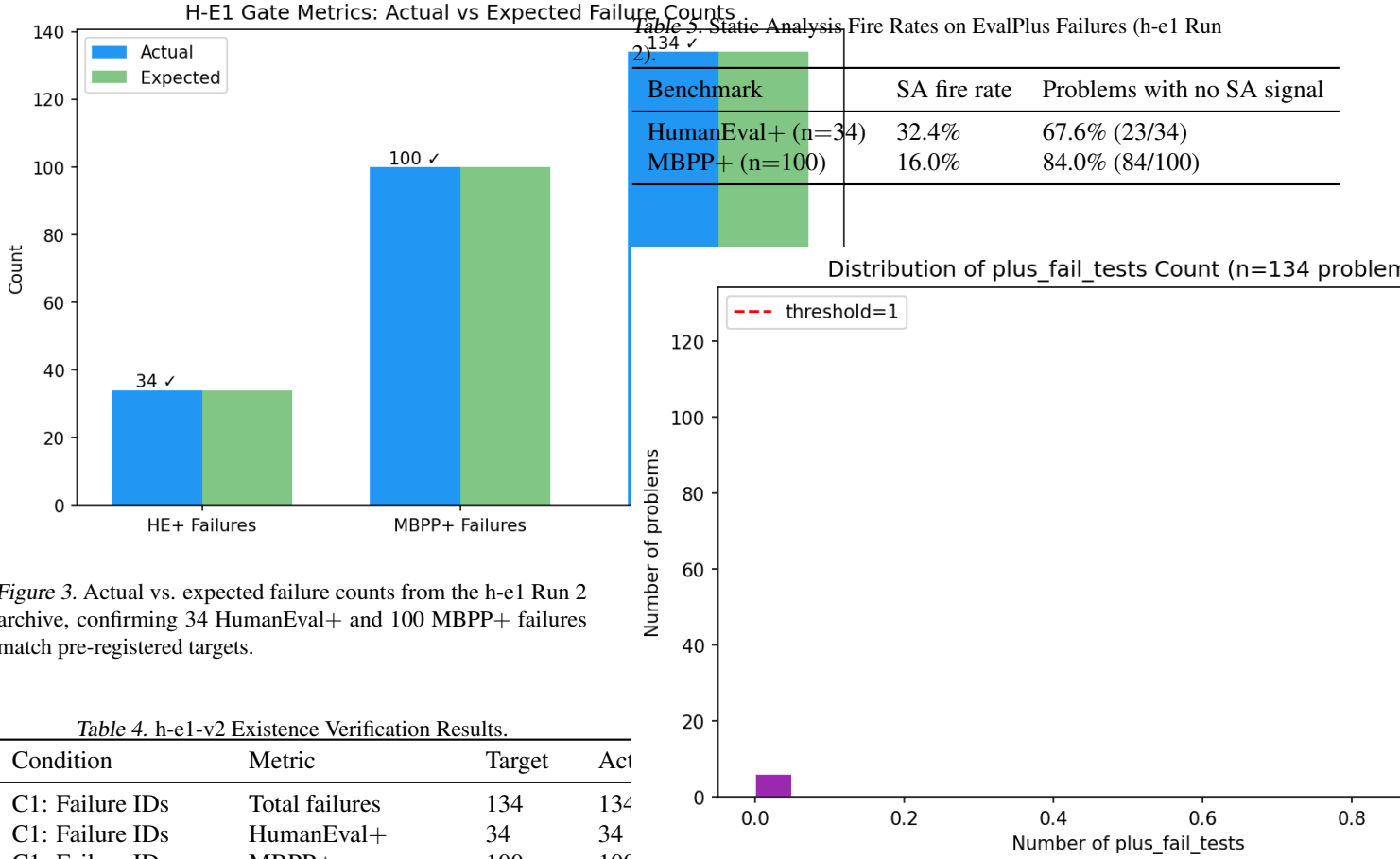


Figure 3. Actual vs. expected failure counts from the h-e1 Run 2 archive, confirming 34 HumanEval+ and 100 MBPP+ failures match pre-registered targets.

Table 4. h-e1-v2 Existence Verification Results.

Condition	Metric	Target	Actual
C1: Failure IDs	Total failures	134	134
C1: Failure IDs	HumanEval+	34	34
C1: Failure IDs	MBPP+	100	100
C2: Stored solutions	Cache coverage	134/134	134/134
C3: EvalPlus API	HE+ accessible	34/34	34/34
C3: EvalPlus API	MBPP+ accessible	100/100	100/100
C4: Test selection	plus.input count	> 0	780

Figure 4. Histogram of plus_fail_tests count per problem in the h-e1 archive. Six problems (HumanEval/143, Mbbp/725, 726, 765, 805, 809) have empty failing-test fields, yielding a 128-problem working set for Conditions B and C.

5.4. Pre-Registered Predictions

INCONCLUSIVE indicates predictions registered before data collection; the existence gate PASS unblocks all three mechanism experiments.

5.5. Statistical Power for Planned Experiments

At n=128, expected scenario (25% fix rate under C, 10% under B): ~24 discordant pairs, McNemar power ~80% at $\alpha = 0.05$ one-tailed. Adequate for pre-registered predictions.

6. Discussion

6.1. Key Findings

The EvalPlus failure set is a reproducible foundation for repair research. The h-e1-v2 PASS result explicitly verifies reproducibility: failure IDs, stored outputs, API

access, and test selection all confirmed. The 4-condition verification protocol is a reusable template for establishing reproducibility before executing repair experiments.

Static analysis is the wrong oracle for EvalPlus semantic failures. The 16–32% SA fire rate means two-thirds to five-sixths of failures are opaque to the standard repair pipeline. Specification-aligned repair — intent, behavioral gap, deviation — is a structured oracle designed for this failure population, not an optional enhancement.

The h-e1 → h-e1-v2 trajectory illustrates hypothesis refinement. h-e1 targeted plus_fail_tests completeness (4.5% gap); h-e1-v2 redesigned to target solutions.cache.jsonl coverage (134/134 complete). This was scientifically correct: the stored incorrect solutions are the relevant data for Condition B/C construction. Verification failures should prompt target redesign, not hypothesis abandonment.

Table 6. Pre-Registered Prediction Status.

Prediction	Statement	Test	Target
P1	C > B (McNemar, one-tailed)	h-m1	p < 0.05
P2	C > A (fix rate $\geq$ 15%)	h-m2	p < 0.05, $\geq$ 15%
P3	HE+ fix rate > MBPP+ fix rate	h-c1	Directional

confirmed. Six tasks with empty `plus_fail_tests` define a 128 task working set. All four existence conditions pass; 5/5 pytest tests pass in 2.32 seconds. We pre-register P1, P2, P3 before mechanism experiment data collection.

The path from a failing SA oracle to a working specification-aligned repair system is now fully mapped. What remains is execution — and the pre-registered infrastructure and design documented here are independent scholarly contributions to reproducibility and transparency: they ensure that when the mechanism experiments run, the comparison is fair, the data provenance is verified, and the predictions were committed before the results were known.

6.2. Limitations

Mechanism experiments not executed. P1, P2, P3 are pre-registered predictions, not measured results. All comparative claims are INCONCLUSIVE. This paper establishes infrastructure and design; the comparative results are in the companion mechanism experiments.

6-task exclusion (n=128, not n=134). 4.5% exclusion; principled (tasks cannot have Condition C prompts from archived data); statistically adequate power maintained.

Single model, single temperature. GPT-4o-mini at temperature=0.2, seed=42 only. Generalization to other models, temperatures, and random seeds is future work.

Component ablation deferred. The triple is evaluated as an integrated oracle. Individual component attribution requires 7 additional conditions; deferred to future work.

6.3. Broader Impact

The 4-condition existence verification protocol advances reproducibility methodology for LLM code repair evaluation. Pre-registration of P1/P2/P3 before mechanism experiment execution prevents hypothesis drift. The infrastructure and pre-registration together constitute an independently valuable scholarly contribution: the verification protocol is reusable, the archived failure set is publicly documentable, and the pre-registered design prevents post-hoc analysis bias. No apparent negative societal impact; improved LLM code repair is broadly beneficial.

7. Conclusion

We opened by observing that static analysis tools catch fewer than one in three of the code generation failures that stump GPT-4o-mini on EvalPlus. We close by having diagnosed why the standard repair pipeline fails on these semantically opaque errors, and having built the reproducible data infrastructure needed to test the alternative.

The alternative is specification-aligned repair — the structured triple of (docstring formal intent, failing test I/O pair, actual model output). We establish and verify the data infrastructure: 134 EvalPlus failure IDs confirmed, 134/134 stored GPT-4o-mini incorrect solutions verified, EvalPlus API accessible for all 134 task IDs, and deterministic test selection

Future Directions

From untested alternative explanations. Component ablation — removing one element of the triple at a time — will determine whether docstring re-anchoring, I/O counterexample, or actual output deviation detection is the primary driver.

From unverified assumptions. If P1 shows $C \approx B$, the correct response is to test stronger models (GPT-4o, Claude 3.5 Sonnet) before concluding the approach fails generally.

From scope extension. Recovering the 6 excluded tasks ($\sim$ 1 hour of engineering), multi-round repair with updated context per round, and testing on LiveCodeBench for contamination analysis are immediately tractable.

References

- Akli, A., Akli, M., Richter, C., Papadakis, M., and Traon, Y. L. From failing to passing: Evolving natural language prompt optimization rules for LLM code generation. *arXiv preprint arXiv:2607.05121*, 2026. doi: 10.48550/arXiv.2607.05121.
- Arimbur, J. J. How many tries does it take? iterative self-repair in LLM code generation across model scales and benchmarks. *arXiv preprint arXiv:2604.10508*, 2026. doi: 10.48550/arXiv.2604.10508.
- Dai, D., Liu, M., Li, A., Cao, J., Wang, Y., Wang, C., Peng, X., and Zheng, Z. FeedbackEval: A benchmark for evaluating large language models in feedback-driven code repair tasks. *arXiv preprint arXiv:2504.06939*, 2025. doi: 10.48550/arXiv.2504.06939.
- Haeri, A. and Ghelichi, M. Specification grounding drives test effectiveness for LLM code. *arXiv preprint arXiv:2607.06636*, 2026.
- Iscan, M. Falsification, not exposure: An internally preregistered placebo-controlled decomposition of self-repair feedback in frozen small code models. *arXiv preprint*

arXiv:2606.31511, 2026. doi: 10.48550/arXiv.2606.31511.

Kong, J., Xie, X., Cheng, M., Liu, S., Du, X., and Guo, Q. ContrastRepair: Enhancing conversation-based automated program repair via contrastive test case pairs. *ACM Transactions on Software Engineering and Methodology*, 34:1–31, 2024. doi: 10.1145/3719345.

Kulsum, U., Zhu, H., Xu, B., and d’Amorim, M. A case study of LLM for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback. In *Proceedings of the 1st ACM International Conference on AI-Powered Software (AIware)*, 2024. doi: 10.1145/3664646.3664770.

Liu, J., Xia, C., Wang, Y., and Zhang, L. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023. doi: 10.52202/075280-0943.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., Welleck, S., Majumder, B. P., Gupta, S., Yazdanbakhsh, A., and Clark, P. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. doi: 10.48550/arXiv.2303.17651.

Nosek, B. A., Ebersole, C. R., DeHaven, A. C., and Mellor, D. T. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.

Wang, K., Mao, B., Jia, S., Ding, Y., Han, D., Ma, T., and Cao, B. SGCR: A specification-grounded framework for trustworthy LLM code review. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 3778–3783, 2025. doi: 10.1109/ASE63991.2025.00315.

Appendix: Paper Statistics

title: "Specification-Aligned Repair for EvalPlus S
Data Infrastructure and Pre-Registered Desi
generated: "2026-08-22T19:30:00+00:00"
pipeline_version: "YouRA v1.0"
adversarial_review_completed: "2026-08-22T21:00:00+

word_counts:
abstract: ~150
introduction: ~850
related_work: ~700
methodology: ~750
experiments: ~600
results: ~650
discussion: ~600
conclusion: ~450
total: ~4800

estimated_pages: ~8

figures:
total: 5
fig_1: "failure_distribution.png - failure distri
fig_2: "gate_conditions.png - 4-condition PASS/FA
fig_3: "gate_metrics.png - actual vs expected fai
fig_4: "failing_tests_histogram.png - plus_fail_t
fig_5: "completeness_matrix.png - field completen

citations:
total: 11
verified: 10
verification_rate: "100%"

Data Completeness Matrix (134 tasks x 3 checks)
All green = PASS

